

# Evaluating Cryptographic API Misuse Detectors for Go

Vivi Andersson, Martin Monperrus

KTH Royal Institute of Technology, Stockholm, Sweden

April 18, 2026

Rio de Janeiro, Brazil

SVM 2026, co-located with ICSE 2026

# Problem Context

- Go is widely used in security-critical infrastructure, including certificate services, container platforms, and cloud tooling.
- Cryptographic API misuse remains a high-impact vulnerability class: the code may compile and pass tests while silently violating security assumptions.
- Yet, unlike Java, the landscape of cryptographic misuse detection for Go has not been systematically characterized.

# Evaluation Challenge

- Existing Go detectors do not expose a common target: they differ in supported rules, reporting granularity, and execution robustness.
- As a result, alert counts are difficult to interpret across tools: more findings may indicate broader coverage, noisier heuristics, or simply a different rule set.
- What is missing is a comparative view that separates tool scope from tool agreement on real Go codebases.

# Prior Work and the Gap

- Comparative work already exists for Java crypto misuse detection, including multi-tool evaluation and false-positive analysis [1, 2, 3].
- Go-specific detector work also exists: *CryptoGo* introduced the first Go detector, and *GOPHER* later extended that line of work [4, 5].
- However, prior Go work does not provide a systematic comparison across the currently available detectors used in practice.

## This paper

We compare the currently available Go crypto misuse detectors, instead of studying only one tool at a time.

[1] Ami et al. "Why Crypto-detectors Fail: A Systematic Evaluation of Cryptographic Misuse Detection Techniques." SP 2022.

[2] Zhang et al. "Automatic Detection of Java Cryptographic API Misuses: Are We There Yet?" TSE 2022.

[3] Chen et al. "Towards Precise Reporting of Cryptographic Misuses." NDSS 2024.

[4] Li et al. "Cryptogo: Automatic Detection of Go Cryptographic API Misuses." ACSAC 2022.

[5] Zhang et al. "Gopher: High-Precision and Deep-Dive Detection of Cryptographic API Misuse in the Go Ecosystem." CCS 2024.

# Research Questions

- **RQ1.** What is the design space of state-of-the-art tools for detecting cryptographic API misuses in Go?
- **RQ2.** To what extent are crypto misuse detection tools for Go consistent with each other?

## Study Design

RQ1 compares tool scope, supported rules, and detection approach.

RQ2 runs all tools on 328 security-relevant Go projects and compares execution coverage, findings, and overlap.

## Contribution

First systematic comparison of four Go cryptographic misuse detectors across 14 rule classes and 328 security-relevant projects.

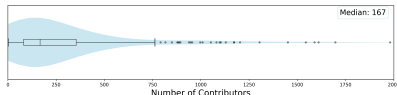
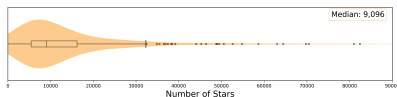
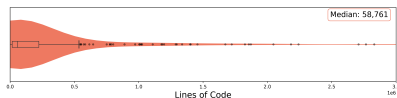
# RQ1: Taxonomy and Tool Support

| Category                 | IDs   | Rules                                                             |
|--------------------------|-------|-------------------------------------------------------------------|
| Cryptographic primitives | 01–03 | insecure algorithms; insecure PRNG; deprecated Go function        |
| Key management           | 04–06 | constant/predictable key; short key length; static/predictable IV |
| Password-based KDF       | 07–09 | short salt; predictable salt; low iterations                      |
| Transport security       | 10–11 | HTTP protocol; TLS/SSL issues                                     |
| Secure Shell             | 12–13 | insecure SSH suite; no host key validation                        |
| Token auth               | 14    | no JWT verification                                               |

- Only rules 02, 05, and 11 are supported by all four tools.
- GOPHER covers the broadest part of the taxonomy, while CODEQL alone supports rule 14.

# Dataset and Scope

- 328 security-relevant open-source Go repositories.
- Repository size is strongly right-skewed, with a median of 58,761 LOC.
- Median popularity is 9,096 stars and median contributor count is 167.



## RQ2: Execution Coverage Across Tools

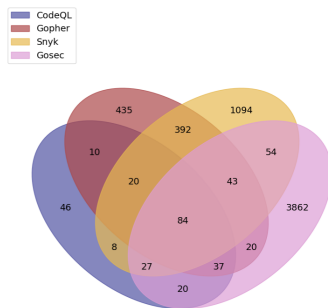
|                                      | CodeQL | Gopher | Gosec | Snyk |
|--------------------------------------|--------|--------|-------|------|
| <b>Projects analyzed (%)</b>         | 91.8   | 77.1   | 100   | 100  |
| <b>With findings (% of analyzed)</b> | 30.9   | 64.8   | 84.5  | 92.7 |
| <b>With findings (% of dataset)</b>  | 28.4   | 50.0   | 84.5  | 92.7 |

### Interpretation

SNYK CODE and GOSSEC analyze all 328 projects, while GOPHER fails on 75 projects and CODEQL on 27. So even before comparing findings, the tools are operating on different subsets of projects.

## RQ2: Cross-Tool Agreement Is Low

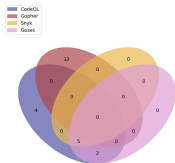
- 7,473 detections collapse to 6,152 unique reported locations.
- Only 1.3% of detections are shared by all four tools.
- Most findings are tool-specific: 92.6% of Gosec detections and 63.5% of SNYK CODE detections are unique locations.



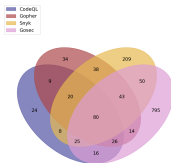
**Figure:** Overall overlap across tools shows that much of the detection volume is tool-unique.

## RQ2: Agreement Depends on the Misuse Rule

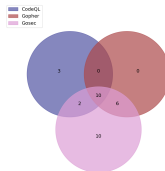
- Rule #05: all four tools support it, but they still disagree sharply.
- Rule #11: the largest class, with 2,033 detections total and 1,049 from GOSec alone.
- Rule #13: only 59 detections total, making manual inspection feasible.



Rule 05



Rule 11



Rule 13

### RQ2 takeaway

Agreement varies substantially by misuse class.

# Manual Analysis

- For Rule #05, overlap is not automatically correctness: 4 of 5 detections shared by CODEQL, GOSEC, and SNYK CODE but missed by GOPHER are in examples or mock code.
- For Rule #11, 3 of 5 sampled GOSEC-only detections are false positives; in our SNYK CODE sample, we found no clear false positives, and the alerts better reflected actual insecure TLS settings.
- For Rule #13, CODEQL contributes unique SSH findings beyond InsecureIgnoreHostKey by reasoning about custom callbacks that return `nil`.

## Interpretation

Cross-tool overlap helps, but the more important result is that the tools differ in what they consider a misuse and how much context they understand.

# Implications for Practice and Evaluation

- **For practitioners:** using a single detector is likely to leave blind spots; using multiple tools is justified by the low overlap.
- **For evaluators:** raw finding counts are misleading when tools disagree this much about the same rule class.
- **For tool builders:** the tools differ in kind, not just in degree: GOPHER covers the broadest rule set, SNYK CODE and GOSSEC run most reliably, and CODEQL contributes higher-overlap alerts plus distinct SSH findings.

The comparative question matters because these detectors vary enough in scope and agreement to change both practical use and experimental conclusions.

# Takeaways

- The main result is not a winner, but a characterization: current Go crypto misuse detectors occupy different points in the design space.
- Coverage, reliability, and agreement vary enough that detector outputs should be interpreted as complementary rather than interchangeable.
- For this problem, evaluating detectors requires comparing what they are able to detect, not only how much they report.

- Future work should move beyond syntax-level alerts toward application-aware security relevance, potentially using AI to infer threat models and deployment context.
- Extend the comparison as tools evolve and new Go misuse classes become detectable.

Thank you

Questions, discussion, and criticism are welcome.